Solution: Making A Large Island

Problem Statement

Given a 2D binary matrix of size `n x n`, return the integer representing the size of the `largest island` after changing at most `0` to `1`.

Here, `Island` is defined as a `4-directionally` connected group of `1`s.

Examples

- **Example 1**:
  - **Input**:

    ```
    [[1, 0, 1, 0],
     [0, 1, 1, 0],
     [0, 0, 0, 1],
     [1, 1, 0, 0]]
    ```

  - **Expected Output**: `6`
  - **Justification**: Converting the water cell at position (2,1) will connect two large islands (top right and bottom left) with a new land cell, creating an island of size 6.
- **Example 2**:
  - **Input**:

    ```
    [[1, 1, 1],
     [1, 0, 0],
     [0, 0, 1]]
    ```

  - **Expected Output**: `6`
  - **Justification**: Changing the cell (1,2) into land will connect the top horizontal island with the bottom right cell, resulting in a single island of size 6.
- **Example 3**:
  - **Input**:

    ```
    [[0, 1, 0],
     [1, 0, 1],
     [0, 1, 0]]
    ```

  - **Expected Output**: `5`
  - **Justification**: By converting the center cell (1,1) to land, all surrounding cells will connect, forming a cross-shaped island with 5 cells.

Solution

To solve this problem, we will first identify and label each connected land area with a unique identifier. For this, a depth-first search (DFS) is suitable as it can traverse through connected land cells effectively. By labeling each land area, we can quickly determine the size of each island and store this information.

After labeling, we will check each water cell in the grid. For each water cell, we will look at its neighboring cells and calculate the potential size of the island if this water cell is converted to land. The key here is to ensure that we only consider unique neighboring islands to avoid counting the same land area multiple times. By doing so, we can accurately determine the maximum island size that can be achieved by converting a single water cell to land. This approach is efficient as it reduces the problem to identifying and summing up unique neighboring islands around water cells.

Step-by-step Algorithm

- **Initialization**:
  - Create a map to store the size of each island, indexed by a unique identifier.
  - Set a variable `maxIslandSize` to 0, which will keep track of the maximum island size found.
  - Use a 2D array `islandId` to store the identifier for each cell in the grid.
  - Set an identifier `id` starting from 2 (since 0 and 1 are already used in the grid).
- **Labeling Islands**:
  - Iterate over each cell in the grid.
  - When a land cell (value 1) is encountered that has not yet been assigned an identifier:
    - Perform a DFS from this cell to label the entire connected island with the current `id`.
    - Increment `id` for the next island.
- **DFS Function**:
  - The DFS function labels all connected land cells with the same identifier.
  - It also counts the number of cells in the island and updates the map with the size of the island.
- **Calculating Potential Island Size**:
  - Iterate over the grid again, this time focusing on water cells (value 0).
  - For each water cell, check its four adjacent cells (up, down, left, right).
  - Maintain a set to store unique island identifiers of neighboring land cells.
  - Calculate the potential size of the island by converting the current water cell to land. This is done by summing the sizes of unique neighboring islands and adding 1 (for the current water cell).
  - Update `maxIslandSize` if this calculated size is larger.
- **Final Step**:
  - After iterating through all cells, return `maxIslandSize` as the result.

Algorithm Walkthrough

Let's walk through the algorithm using Example 1, where the input grid is:

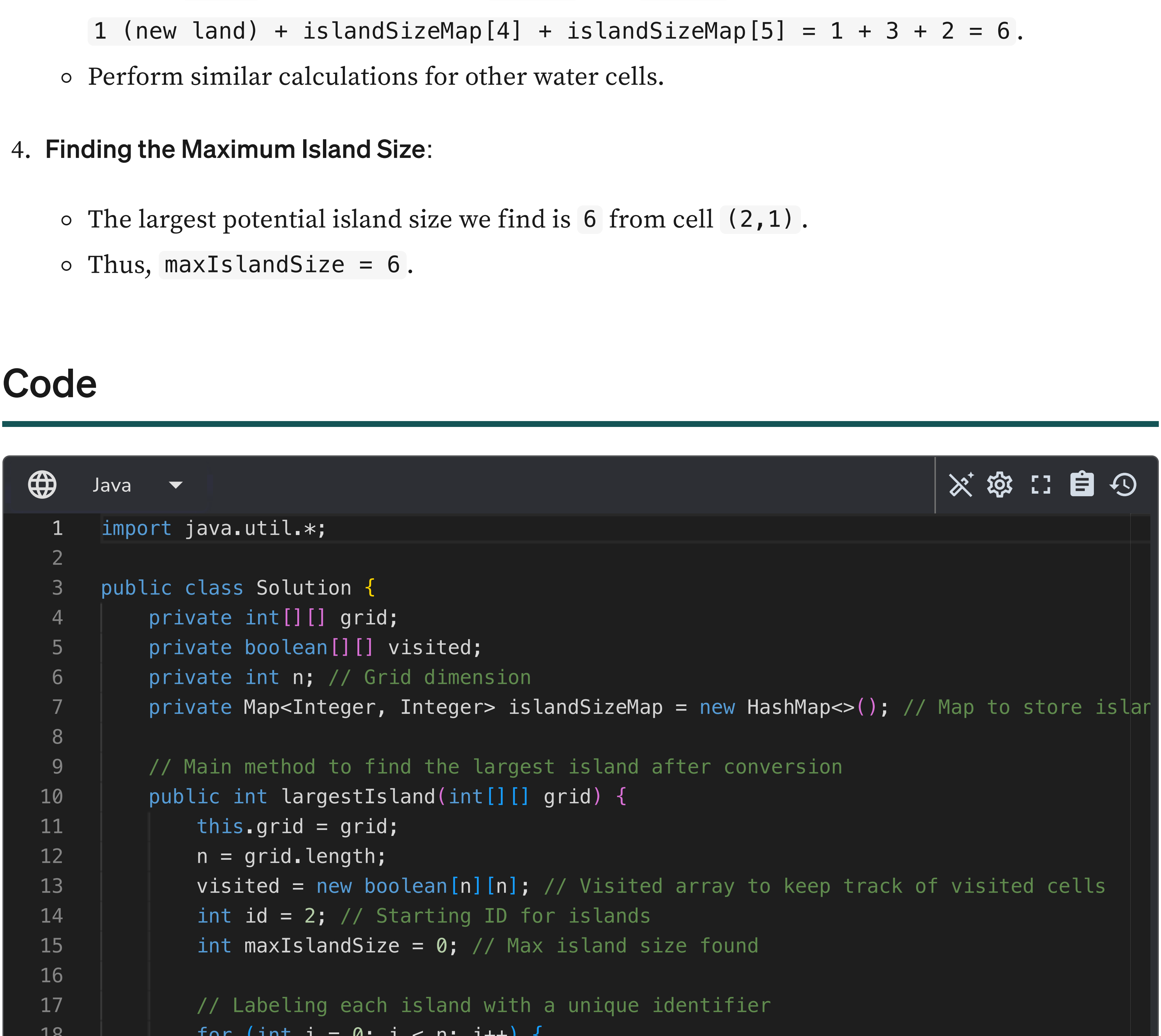
```
[[1, 0, 1, 0],
 [0, 1, 1, 0],
 [0, 0, 0, 1],
 [1, 1, 0, 0]]
```

1. **Initialize Data Structures**:
   - The grid is a `4x4` matrix.
   - We use a map `islandSizeMap` to track the size of each island.
   - An `islandId` matrix stores the identifier of each island.
   - Initialize `maxIslandSize` as 0 and `id` as 2.
2. **Labeling Islands with DFS**:
   - Start iterating over the grid.
   - First land cell is at `(0,0)`. Perform DFS, label it with `id = 2`. This is a single cell island; `islandSizeMap[2] = 1`.
   - Next land at `(0,2)`, gets `id = 3`. `islandSizeMap[3] = 3`.
   - The next unvisited land starts at `(2,3)`. Label these connected cells with `id = 4`. This island has 1 cell; `islandSizeMap[4] = 1`.
   - Finally, label the bottom left island starting at `(3,0)` with `id = 5`. This island has 2 cells; `islandSizeMap[5] = 2`.
   - The `grid` now looks like:

     ```
     [[2, 0, 3, 0],
      [0, 3, 3, 0],
      [0, 0, 0, 4],
      [5, 5, 0, 0]]
     ```

3. **Calculating Potential Island Sizes**:
   - Examine each water cell to determine the potential island size if converted to land.
   - For cell `(0,1)`, we have neighboring islands with `id = 2` and `id = 3`. Potential size = `1 (new land) + islandSizeMap[2] + islandSizeMap[3] = 1 + 1 + 3 = 5`.
   - For cell `(0,3)`, we have neighboring islands with `id = 3`. Potential size = `1 (new land) + islandSizeMap[3] = 1 + 3 = 4`.
   - For cell `(1,0)`, we have neighboring islands with `id = 3` and `id = 4`. Potential size = `1 (new land) + islandSizeMap[3] + islandSizeMap[4] = 1 + 3 + 1 = 5`.
   - For cell `(1,3)`, we have neighboring islands with `id = 2` and `id = 3`. Potential size = `1 (new land) + islandSizeMap[2] + islandSizeMap[3] = 1 + 1 + 3 = 5`.
   - For cell `(2,0)`, we have neighboring islands with `id = 5`. Potential size = `1 (new land) + islandSizeMap[5] = 1 + 2 = 3`.
   - For cell `(2,1)`, the neighbors are `id = 4` and `id = 5`. Potential size = `1 (new land) + islandSizeMap[4] + islandSizeMap[5] = 1 + 3 + 2 = 6`.
   - Perform similar calculations for other water cells.
4. **Finding the Maximum Island Size**:
   - The largest potential island size we find is `6` from cell `(2,1)`.
   - Thus, `maxIslandSize = 6`.

Code



```java
import java.util.*;

public class Solution {
    private int[][] grid;
    private boolean[][] visited;
    private int n; // Grid dimension
    private Map<Integer, Integer> islandSizeMap = new HashMap<>(); // Map to store islar

    // Main method to find the largest island after conversion
    public int largestIsland(int[][] grid) {
        this.grid = grid;
        n = grid.length;
        visited = new boolean[n][n]; // Visited array to keep track of visited cells
        int id = 2; // Starting ID for islands
        int maxIslandSize = 0; // Max island size found

        // Labeling each island with a unique identifier
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == 1 && !visited[i][j]) {
                    int size = dfs(i, j, id); // Perform DFS to find island size
```

You must run your code first

Complexity Analysis

Time Complexity:

- **DFS Traversal**: `O(N²)`, where N is the grid dimension. DFS is performed for each cell in the grid.
- **Calculating Potential Island Size**: Also `O(N²)`, as it involves checking adjacent cells for each grid cell.
- **Overall Time Complexity**: `O(N²)`, since both steps are linear with respect to the number of cells.

Space Complexity:

Space used for the `visited` 2D array and the `islandSizeMap`, both of which are proportional to the grid size.

- **DFS Stack Space**: `O(N²)` in the worst case (for a fully connected grid).
- **Overall Space Complexity**: `O(N²)`, primarily due to the 2D array and the recursion stack space.